

Học Sâu (Deep Learning)

Chương 18: Kỹ năng Thực chiến & Tối ưu hóa Mô hình

Đại học Đông Á

August 14, 2026

1. Tối ưu hóa Siêu tham số (Hyperparameter Tuning)
2. Suy diễn Đám đông - Model Ensembling
3. Huấn luyện Phân tán (Distributed Training)
4. Tối ưu Bộ nhớ: Mixed-Precision & Quantization
5. Tổng kết: Vòng lặp Tiến bộ

Tham số và Siêu tham số

- **Tham số (Parameters):** Là các trọng số (Weights) và độ lệch (Biases) mà mô hình tự động học được thông qua thuật toán Gradient Descent và Backpropagation.
- **Siêu tham số (Hyperparameters):** Là các thiết lập kiến trúc do con người định nghĩa trước khi huấn luyện. Ví dụ: Số lớp (layers), số node ẩn, Learning Rate, kích thước Batch, hàm Kích hoạt.
- **Vấn đề:** Không gian Siêu tham số không liên tục và không thể đạo hàm, do đó không thể dùng Gradient Descent để tối ưu.

Các chiến lược tìm kiếm Siêu tham số

- **Grid Search (Tìm kiếm dạng lưới):** Duyệt qua mọi tổ hợp có thể của các siêu tham số. Tuyệt đối chính xác nhưng tốn thời gian theo cấp số nhân (Combinatorial Explosion).
- **Random Search (Tìm kiếm ngẫu nhiên):** Chọn ngẫu nhiên các tổ hợp trong không gian định sẵn. Nhanh hơn Grid Search và thường tìm được cấu hình đủ tốt trong thời gian ngắn.
- **Bayesian Optimization (Tối ưu hóa Bayes):** Sử dụng mô hình thống kê (Surrogate Model) để học từ các lần thử nghiệm trước, từ đó dự đoán tọa độ siêu tham số nào có khả năng cho ra kết quả tốt nhất ở lần thử tiếp theo.

- Trong thực tế, Kỹ sư Machine Learning không điều chỉnh siêu tham số bằng tay. Họ sử dụng thư viện tự động như **KerasTuner**.
- Quy trình hoạt động của KerasTuner:
 - ➊ Định nghĩa Không gian tìm kiếm (Search Space) (Ví dụ: Số lớp từ 2 đến 5, Learning rate $\{0.01, 0.001\}$).
 - ➋ Xác định Hàm mục tiêu (Objective) (Ví dụ: `val_accuracy` cao nhất).
 - ➌ Chạy thuật toán dò tìm (Random Search, Hyperband, hoặc Bayesian Optimization).
 - ➍ Lấy ra mô hình tốt nhất để triển khai.

Ứng dụng KerasTuner cho Kiến trúc SOTA

- KerasTuner không chỉ tinh chỉnh các mạng đơn giản mà còn hỗ trợ không gian tìm kiếm được xây dựng sẵn cho các kiến trúc đỉnh cao (State-of-the-Art).
- Ví dụ: `kt.applications.HyperXception` và `kt.applications.HyperResNet`.
- Bằng cách sử dụng các mô hình Hyper này, hệ thống sẽ tự động tìm kiếm siêu tham số tối ưu trên nền tảng của ResNet và Xception mà không cần bạn phải tự thiết kế lại các khối Convolution phức tạp.

Mã nguồn: Truy vấn cấu hình và Epoch tốt nhất

```
1 def get_best_epoch(hp):
2     model = build_model(hp)
3     callbacks = [
4         keras.callbacks.EarlyStopping(
5             # High patience to ensure convergence
6             monitor="val_loss", mode="min", patience=10
7         )
8     ]
9     history = model.fit(
10         x_train, y_train, validation_data=(x_val, y_val),
11         epochs=100, batch_size=128, callbacks=callbacks,
12     )
13     val_loss_per_epoch = history.history["val_loss"]
14     best_epoch = val_loss_per_epoch.index(min(val_loss_per_epoch)) + 1
15     return best_epoch
```

Mã nguồn: Huấn luyện mô hình siêu tham số tối ưu

```
1 def get_best_trained_model(hp):
2     best_epoch = get_best_epoch(hp)
3     model = build_model(hp)
4     # Train on the full dataset without validation split
5     model.fit(
6         x_train_full, y_train_full, batch_size=128, epochs=int(best_epoch * 1.2)
7     )
8     return model
9
10 best_models = []
11 for hp in best_hps:
12     model = get_best_trained_model(hp)
13     model.evaluate(x_test, y_test)
14     best_models.append(model)
```


Rủi ro: Overfitting trên Validation Set

- Việc tinh chỉnh Siêu tham số có một cạm bẫy cực kỳ nguy hiểm: **Quá khớp tập xác thực (Overfitting Validation Set)**.
- Vì ta luôn chọn những Siêu tham số đem lại kết quả tốt nhất trên tập Validation, ta đã vô tình biến tập Validation thành một dạng dữ liệu dùng để học.
- **Hậu quả:** Khi ra thực tế, mô hình sẽ hoạt động kém hơn mong đợi.
- **Giải pháp:** Luôn giữ lại một tập Test hoàn toàn độc lập (Không được dùng tập Test này để Early Stopping hay chọn KerasTuner).

Triết lý Ensemble: Suy diễn Đám đông

- **Ensembling** (Tổ hợp mô hình) là kỹ thuật gộp dự đoán của nhiều mô hình khác nhau để tạo ra một dự đoán cuối cùng mạnh mẽ hơn.
- Triết lý: "Nhiều mô hình yếu cộng lại thành một mô hình mạnh".
- Dựa trên nguyên lý Thống kê: Nếu các mô hình có lỗi độc lập với nhau, sai số tổng thể sẽ triệt tiêu lẫn nhau.
- Điều này giống như câu chuyện "Thầy bói xem voi" - mỗi thầy bói chỉ sờ được một bộ phận, nhưng nếu họ hợp lại và tổng hợp ý kiến, bức tranh toàn cảnh về con voi sẽ chính xác.

Mã nguồn: Kỹ thuật Averaging (Trung bình hóa dự đoán)

```
1 # Uses four different models to compute initial predictions
2 preds_a = model_a.predict(x_val)
3 preds_b = model_b.predict(x_val)
4 preds_c = model_c.predict(x_val)
5 preds_d = model_d.predict(x_val)
6
7 # Simple average
8 # This new prediction array should be more accurate than any of the initial ones.
9 final_preds = 0.25 * (preds_a + preds_b + preds_c + preds_d)
```

Mã nguồn: Kỹ thuật Trung bình có Trọng số (Weighted Average)

```
1 # Not all models are equally good, so we weight them based on validation score
2 preds_a = model_a.predict(x_val)
3 preds_b = model_b.predict(x_val)
4 preds_c = model_c.predict(x_val)
5 preds_d = model_d.predict(x_val)
6
7 # These weights (0.5, 0.25, 0.1, 0.15) are assumed to be learned empirically.
8 final_preds = 0.5 * preds_a + 0.25 * preds_b + 0.1 * preds_c + 0.15 * preds_d
```

Đa dạng hóa - Chìa khóa của Ensembling

- Trong thuật toán tổ hợp, nếu ta kết hợp 5 mô hình giống hệt nhau, ta sẽ chẳng đạt được cải thiện nào.
- **Diversity (Đa dạng):** Các mô hình phải học từ những góc nhìn (Representations) khác nhau. Ví dụ: Kết hợp 1 CNN, 1 Transformer, và 1 Random Forest.
- **Bất đẳng thức Jensen:** Nếu hàm mất mát là hàm lồi (Convex), sai số của mô hình Ensemble luôn nhỏ hơn hoặc bằng trung bình sai số của các mô hình thành phần:

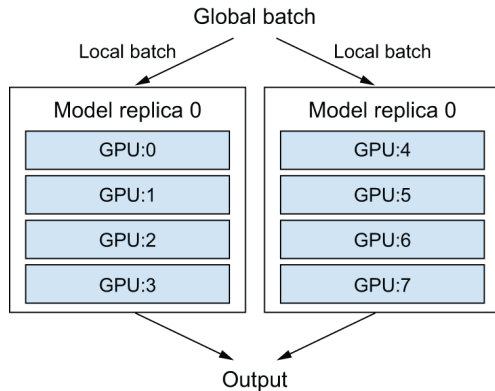
$$L(\mathbb{E}[M_i]) \leq \mathbb{E}[L(M_i)]$$

Tại sao cần Huấn luyện phân tán?

- Các mô hình SOTA (Ví dụ: LLMs, Vision Transformers) mất tới hàng chục nghìn năm để huấn luyện nếu chỉ dùng 1 GPU.
- Bằng cách nối mạng hàng nghìn GPU/TPU với nhau thành một cụm siêu máy tính (Cluster), thời gian huấn luyện giảm xuống chỉ còn vài tuần.
- Vấn đề cốt lõi: Làm sao chia việc cho hàng nghìn GPU chạy đồng thời mà không bị xung đột bộ nhớ và đảm bảo tiến trình Gradient cập nhật đúng?

Phân tán Dữ liệu vs Phân tán Mô hình

- **Data Parallelism:** Mỗi GPU chứa 1 bản sao y hệt của mô hình. Dữ liệu (Batch) được chia nhỏ ra (Ví dụ Batch 1024 chia thành 8 phần cho 8 GPU).
- **Model Parallelism:** Khi mô hình quá lớn không nhét vừa RAM 1 GPU, ta phải "chặt đứt" các Layer của mạng Nơ-ron và rải chúng ra nhiều GPU khác nhau.



Hình 18.1: Sơ đồ hệ thống tính toán Phân tán hỗn hợp

Mã nguồn: Khởi tạo Data Parallelism bằng JAX backend

```
1 import keras
2
3 # Get a list of available devices
4 # Example: ["gpu:0", "gpu:1", "gpu:2", "gpu:3"]
5 devices = keras.distribution.list_devices()
6
7 # Initialize Data Parallelism over all GPUs
8 keras.distribution.set_distribution(keras.distribution.DataParallel())
9
10 # Or over specific GPUs
11 keras.distribution.set_distribution(
12     keras.distribution.DataParallel(["gpu:0", "gpu:1"])
13 )
```


Đồng bộ hóa Gradient (All-Reduce Algorithm)

- Trong Data Parallelism, nếu mỗi GPU nhận 1 tập dữ liệu con khác nhau, đạo hàm (Gradient) của chúng tính ra sẽ khác nhau.
- Nếu cập nhật trọng số trực tiếp, 8 GPU sẽ tự tạo ra 8 mô hình khác biệt hoàn toàn!
- **Đồng bộ (Synchronous Training):** Thuật toán All-Reduce ép tất cả GPU phải dừng lại vào cuối mỗi Step, truyền mạng gửi Gradient cho nhau, tính trung bình rồi mới thực hiện cập nhật cục bộ. Do đó, các GPU luôn có trọng số y hệt nhau.

Mã nguồn: API DeviceMesh phân bổ hệ thống GPU

```
1 # We assume eight devices, organized as a 2 x 4 grid.
2 device_mesh = keras.distribution.DeviceMesh(
3     shape=(2, 4),
4     # It's convenient to give your axes meaningful names!
5     axis_names=["data", "model"],
6 )
7
8 # You can also pass explicit devices
9 devices = [f"gpu:{i}" for i in range(8)]
10 device_mesh = keras.distribution.DeviceMesh(
11     shape=(2, 4), axis_names=["data", "model"], devices=devices,
12 )
```

Mã nguồn: Phân vùng LayoutMap (Model Parallelism)

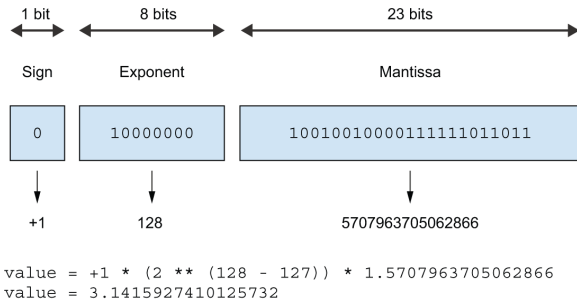
```
1 layout_map = keras.distribution.LayoutMap(device_mesh)
2 # None means "replicate the variable along this dimension."
3 # "model" means shard across the "model" axis of the device mesh.
4 layout_map["sequential/dense/kernel"] = (None, "model")
5 layout_map["sequential/dense/bias"] = ("model",)
6 layout_map["sequential/dense_1/kernel"] = (None, "model")
7 layout_map["sequential/dense_1/bias"] = ("model",)
8
9 model_parallel = keras.distribution.ModelParallel(
10     layout_map=layout_map,
11     batch_dim_name="data", # Axis named "data" handles data parallelism
12 )
13 keras.distribution.set_distribution(model_parallel)
```

Mã nguồn: Tối ưu hoá TPU (Tensor Processing Unit)

```
1 # TPUs are insanely fast but require large batch sizes to stay busy.
2 # If your dataset is small, cache it into memory to prevent IO bottleneck.
3 dataset = dataset.cache()
4
5 # Use "steps_per_execution" to fuse multiple steps into a single TPU run.
6 # This avoids the overhead of passing data back-and-forth to CPU.
7 model.compile(
8     optimizer="adam",
9     loss="sparse_categorical_crossentropy",
10     steps_per_execution=8 # Run 8 batches per call
11 )
```

Biểu diễn dấu phẩy động trong Khoa học Máy tính

- Máy tính lưu số thực IEEE 754 với 3 thành phần: Dấu (Sign), Số mũ (Exponent) và Định trị (Mantissa).
- Các giá trị được biểu diễn cách nhau một độ phân giải không đồng đều. Số càng lớn, khoảng cách giữa 2 số kế tiếp càng thưa.
- Float32:** Dùng 32 bits. Chứa được dải giá trị rất chính xác, nhưng tiêu hao 4 bytes mỗi số.
- Float16 & Bfloat16:** Dùng 16 bits, tiết kiệm một nửa lượng RAM.



Hình 18.2: Độ chính xác dấu phẩy động của số Pi

So sánh cấu trúc Float16 và Bfloat16

- **Float16 (Half Precision):** Sử dụng 5 bits Số mũ và 10 bits Định trị. Rất chính xác cho các số nhỏ, nhưng dải biểu diễn tối đa dễ bị tràn số (Overflow).
- **Bfloat16 (Brain Float):** Sử dụng 8 bits Số mũ và 7 bits Định trị. Bhy sinh độ chính xác chi tiết (Mantissa) để mở rộng dải biểu diễn không lồ tương đương Float32. Bfloat16 hoạt động cực kỳ mượt trên TPU.

dtype	Bit ký hiệu	Bit số mũ	Bit Mantissa
float16	1	5	10
bfloat16	1	8	7
float32	1	8	23

Mã nguồn: Kích hoạt Float16 Inference

```
1 import keras
2
3 # Configure Keras to use float16 by default
4 # This provides nearly 2x speedup on modern GPUs/TPUs during predict()
5 keras.config.set_dtype_policy("float16")
6
7 # Alternatively, for TPUs, bfloat16 is usually better
8 keras.config.set_dtype_policy("bfloat16")
```

Kỹ thuật Mixed-Precision Training

- Trong lúc Training, nếu dùng float16 100%, giá trị Gradient (vốn rất nhỏ $\approx 10^{-6}$) sẽ bị làm tròn về số 0. Quá trình huấn luyện sụp đổ (Underflow).
- **Mixed-Precision:** Kết hợp ưu điểm của cả hai.
- **Forward & Backward Pass:** Hệ thống ép kích hoạt xuống FP16 để tăng tốc siêu hạng quá trình nhân ma trận (Matrix Multiplication).
- **Update Weights:** Master Weights được lưu trữ bằng FP32 để cộng Gradient một cách an toàn.

Mã nguồn: Kích hoạt Mixed-Precision trong Keras

```
1 import keras
2
3 # Turns on Mixed Precision Training automatically
4 keras.config.set_dtype_policy("mixed_float16")
5
6 # To prevent small gradients from rounding down to zero, we multiply the loss
7 # by a large constant scale factor (e.g. 1024.0) during Backpropagation.
8 optimizer = keras.optimizers.Adam(learning_rate=1e-3, loss_scale_factor=10)
9
10 # Better yet, let LossScaleOptimizer figure out the dynamic scaling automatically:
11 optimizer = keras.optimizers.LossScaleOptimizer(
12     keras.optimizers.Adam(learning_rate=1e-3)
13 )
```

- Từ Float16, liệu ta có thể nén mô hình xuống **Float8** (8 bits)?
- Với Float8, tính toán sẽ sai số thảm họa nếu không có các kỹ thuật điều chỉnh Scale bù đắp. Keras chỉ hỗ trợ Float8 cho các lớp đặc thù của Transformer (Dense, EinsumDense, Embedding).
- Float8 thường chỉ hiệu quả với các dòng GPU cực mạnh (NVIDIA H100) và áp dụng cho các siêu mô hình Foundation Models (> 5 Tỷ tham số).

Mất mát thông tin do Lượng tử hóa

- Khi ép từ dải vô hạn của số thực xuống chỉ còn 256 giá trị của Int8, thông tin chắc chắn bị suy giảm.
- **Post-Training Quantization (PTQ):** Huấn luyện xong mô hình bằng Float32 rồi ép kiểu. Đơn giản, nhanh gọn nhưng giảm độ chính xác của mô hình.
- **Quantization-Aware Training (QAT):** Trong lúc huấn luyện Float32, hệ thống mô phỏng việc bị cắt cụt xuống Int8 ở quá trình Forward Pass. Do đó mô hình tự thích nghi với độ nhiễu lượng tử, giữ nguyên độ chính xác khi thực sự triển khai.

Lượng tử hóa (Quantization)

- Để đưa các mô hình AI khổng lồ lên chạy trên Điện thoại hoặc IoT (Edge Devices) cho mục đích Suy diễn (Inference), ta phải nén chúng lại.
- **Quantization:** Ép kiểu các Trọng số từ số thực Float32 xuống số nguyên Int8. Giúp giảm 4 lần dung lượng RAM lưu trữ và chạy cực nhanh trên CPU.
- Thuật toán thường chia tỷ lệ dựa trên Max Absolute Value của phân phối. Các tham số sẽ nằm gọn trong khoảng số nguyên $[-127, 127]$.

Mã nguồn: Mô phỏng thuật toán Abs-Max Lượng tử hoá INT8

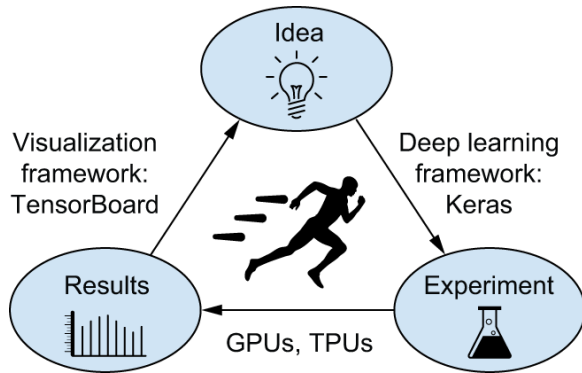
```
1 from keras import ops
2
3 def abs_max_quantize(value):
4     # Max of absolute value of the tensor
5     abs_max = ops.max(ops.abs(value), keepdims=True)
6     # Scale is max of int range (127) divided by max of tensor.
7     scale = ops.divide(127, abs_max + 1e-7)
8
9     scaled_value = value * scale
10    # Rounding and clipping first is more accurate than directly casting.
11    scaled_value = ops.clip(ops.round(scaled_value), -127, 127)
12
13    # Casts to int8
14    scaled_value = ops.cast(scaled_value, dtype="int8")
15    return scaled_value, scale
```

Mã nguồn: API Int8 Quantization trong Keras

```
1 # Instantiates a trained model (or any quantizable layer like Transformer)
2 model = ...
3
4 # Boom! Compress the model.
5 model.quantize("int8")
6
7 # Now predict() and call() will run (partially) in int8!
8 # Very fast on CPU and takes 4x less RAM
9 predictions = model.predict(x_test)
```

Vòng lặp Tiến bộ (The Loop of Progress)

- AI không bao giờ hoàn hảo ngay từ lần Train đầu tiên.
- Một kỹ sư Machine Learning phải liên tục thực thi vòng lặp: **Đánh giá** → **Phân tích lỗi** → **Tinh chỉnh ý tưởng** → **Huấn luyện lại**.
- Do đó, tối ưu Hệ thống (Multi-GPU, Mixed Precision) không chỉ để chạy cho vui, mà nó thu hẹp thời gian mỗi chu kỳ. Giúp ta tạo ra AI thông minh hơn trong cùng một quỹ thời gian.



Hình 18.3: Vòng lặp phát triển vô tận của ML

- **Hyperparameter Tuning:** Dùng công cụ KerasTuner để tự động dò tìm kiến trúc, tránh điều chỉnh thủ công mù quáng.
- **Ensembling:** Áp dụng Bất đẳng thức Jensen và Suy diễn đám đông để đẩy hiệu suất mô hình vượt qua giới hạn của 1 mạng duy nhất.
- **Distributed Training:** Làm chủ Data Parallelism, DeviceMesh và LayoutMap JAX để thống trị phần cứng Data Center.
- **Memory Optimization:** Mix-Precision và Lượng tử hóa Int8 là yếu tố sống còn để đưa AI ra Sản phẩm thương mại (Production).
- **Kết thúc giáo trình:** Chúc mừng bạn đã hoàn thành giáo trình Học sâu hiện đại, từ Keras, ResNet, Transformer cho đến AI Sinh học. Chúc các bạn vững bước trên con đường AI Kỹ sư thực chiến!